1.Introduction:

In this problem statement, we are presented with a scenario in the year 2912 where the **Spaceship Titanic** embarked on its maiden voyage, carrying passengers from our solar system to newly habitable exoplanets. However, during its journey, the ship encountered a spacetime anomaly, causing nearly half of the passengers to be transported to another dimension.

This report focuses on the **Spaceship Titanic** dataset, which contains personal records of the passengers aboard the spaceship. The dataset consists of **869 rows** and **12 columns** in the training set and **427 rows** in the test set. The task is to preprocess the data, handle outliers, and prepare it for predictive modeling. The goal is to predict which passengers were transported to another dimension after the anomaly.

The dataset includes features like **HomePlanet**, **CryoSleep**, **Cabin**, **Age**, **Fare**, and others. The analysis will address missing values, data inconsistencies, and outlier detection to ensure the dataset is ready for modeling.

2.Dataset Documentation:

2.1 Dataset Overview

For this report, the dataset chosen is the **Spaceship Titanic** dataset, which is sourced from the **Kaggle competition** titled *"Spaceship Titanic: Predict which passengers were transported to another dimension"*. The dataset contains information about passengers aboard a spaceship and includes various features related to their journey, such as their **origin**, **destination**, **age**, and whether they were successfully **transported** after the ship encountered a spacetime anomaly.

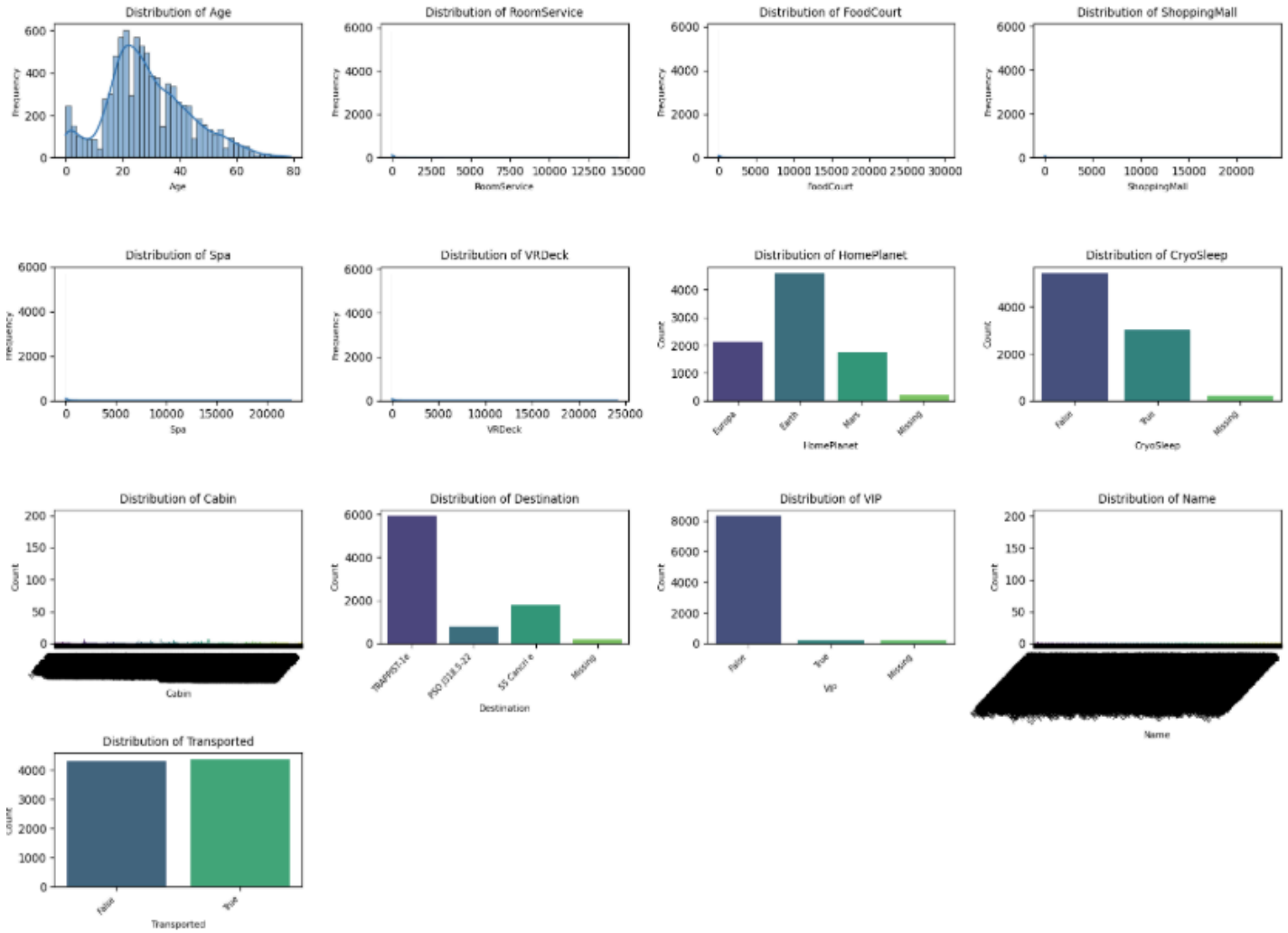
This dataset comprises both **numerical** and **categorical** features that provide insights into the passengers' characteristics and travel details. These features include attributes such as **HomePlanet**, **CryoSleep**, **Cabin**, and **Fare**, among others, making it ideal for predictive modeling tasks.

This section presents a comprehensive visual overview of the dataset before any data cleaning or preprocessing steps were applied. These graphs display the raw distributions of both numerical and categorical features, including the explicit presence of missing values where applicable.

The purpose of this visualization is to:

* Understand the initial characteristics and patterns of the data.
* Identify columns with significant missing values.
* Observe original data types and potential inconsistencies.
* Detect any obvious outliers or unusual distributions in their raw form.

This initial insight is crucial for planning and validating the data preprocessing strategy.



2.2 Dataset Files and Fields

The dataset for this report consists of three files: **`train.csv`**, **`test.csv`**, and **`sample_submission.csv`**.

- train.csv - Personal records for about two-thirds (~8700) of the passengers, to be used as training data.
    - **PassengerId** - A unique Id for each passenger. Each Id takes the form gggg_pp where gggg indicates a group the passenger is travelling with and pp is their number within the group. People in a group are often family members, but not always.
    - **HomePlanet** - The planet the passenger departed from, typically their planet of permanent residence.
    - **CryoSleep** - Indicates whether the passenger elected to be put into suspended animation for the duration of the voyage. Passengers in cryosleep are confined to their cabins.
    - **Cabin** - The cabin number where the passenger is staying. Takes the form deck/num/side, where side can be either P for *Port* or S for *Starboard*.
    - **Destination** - The planet the passenger will be debarking to.
    - **Age** - The age of the passenger.
    - **VIP** - Whether the passenger has paid for special VIP service during the voyage.
    - **RoomService**, **FoodCourt**, **ShoppingMall**, **Spa**, **VRDeck** - Amount the passenger has billed at each of the *Spaceship Titanic*'s many luxury amenities.
    - **Name** - The first and last names of the passenger.
    - **Transported** - Whether the passenger was transported to another dimension. This is the target, the column you are trying to predict.
- test.csv - Personal records for the remaining one-third (~4300) of the passengers, to be used as test data.
- sample_submission.csv - A submission file in the correct format.
    - **PassengerId** - Id for each passenger in the test set.
    - **Transported** - The target. For each passenger, predict either True or False.

3.Approach:Data Analysis Methodology:

Our approach to analyzing the dataset involved a systematic methodology encompassing several key stages. This section outlines the sequential steps taken, from initial data ingestion to the final preparation and preliminary exploration, ensuring the data's quality and readiness for subsequent modeling. Each phase built upon the last, progressively refining the dataset and extracting meaningful insights.

The core steps of our analytical journey include:

3.1 Data Loading & Reading

In this initial phase, the primary goal was to load the raw dataset into a suitable structure for analysis. The pandas library, a powerful and flexible open-source data analysis and manipulation tool, was utilized for this purpose.

Library Used: **import pandas as pd**

Method: pd.read_csv()

Description: The train.csv file was loaded into a pandas DataFrame named train. This step ensured that the dataset was accessible in a tabular format, ready for subsequent inspection and preprocessing.

3.2 Inspecting the Data

Once the data was loaded, a crucial first step was to gain a foundational understanding of its structure, content, and initial characteristics. This involved a series of inspection commands to get a quick overview.

Methods Used:

train.info(): Provided a concise summary of the DataFrame, including the number of entries, column names, non-null counts, and data types for each column. This was essential for identifying potential missing values and inconsistent data types early on.

train.head(): Displayed the first five rows of the DataFrame, offering a glimpse into the actual data values and helping to confirm correct data loading and column interpretation.
train.shape: Returned a tuple representing the dimensions of the DataFrame (number of rows, number of columns), giving a clear idea of the dataset's size.
train.columns: Listed all column names in the DataFrame, ensuring awareness of all available features.

```
••• Data Overview:
    <class 'pandas.core.frame.DataFrame'>
    RangeIndex: 8693 entries, 0 to 8692
    Data columns (total 14 columns):
     #   Column        Non-Null Count  Dtype
    ---  ------        --------------  -----
     0   PassengerId   8693 non-null   object
     1   HomePlanet    8492 non-null   object
     2   CryoSleep     8476 non-null   object
     3   Cabin         8494 non-null   object
     4   Destination   8511 non-null   object
     5   Age           8514 non-null   float64
     6   VIP           8490 non-null   object
     7   RoomService   8512 non-null   float64
     8   FoodCourt     8510 non-null   float64
     9   ShoppingMall  8485 non-null   float64
     10  Spa           8510 non-null   float64
     11  VRDeck        8505 non-null   float64
     12  Name          8493 non-null   object
     13  Transported   8693 non-null   bool
    dtypes: bool(1), float64(6), object(7)
    memory usage: 891.5+ KB
    None
```

Initial Findings: This inspection revealed the presence of various data types (object, float64, bool) and, most importantly, highlighted columns with a notable number of missing values, setting the stage for the preprocessing phase.

```
First Few Rows of the Dataset:
  PassengerId HomePlanet CryoSleep  Cabin  Destination   Age    VIP  \
0     0001_01     Europa     False  B/0/P  TRAPPIST-1e  39.0  False
1     0002_01      Earth     False  F/0/S  TRAPPIST-1e  24.0  False
2     0003_01     Europa     False  A/0/S  TRAPPIST-1e  58.0   True
3     0003_02     Europa     False  A/0/S  TRAPPIST-1e  33.0  False
4     0004_01      Earth     False  F/1/S  TRAPPIST-1e  16.0  False

   RoomService  FoodCourt  ShoppingMall     Spa  VRDeck               Name  \
0          0.0        0.0           0.0     0.0     0.0    Maham Ofracculy
1        109.0        9.0          25.0   549.0    44.0       Juanna Vines
2         43.0     3576.0           0.0  6715.0    49.0      Altark Susent
3          0.0     1283.0         371.0  3329.0   193.0       Solam Susent
4        303.0       70.0         151.0   565.0     2.0  Willy Santantines

   Transported
0        False
1         True
2        False
3        False
4         True

Total Number of Rows and Columns:
Rows: 8693, Columns: 14

Columns in the Dataset:
Index(['PassengerId', 'HomePlanet', 'CryoSleep', 'Cabin', 'Destination', 'Age',
       'VIP', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck',
       'Name', 'Transported'],
      dtype='object')
```

3.3 Data Cleaning & Preprocessing

This phase involved systematically identifying and addressing various data quality issues to prepare the dataset for reliable analysis and modeling. The steps undertaken were as follows:

3.3.1 Checking for Missing Values

We began by identifying all columns containing missing values using the train.isnull().sum() function and calculated their respective percentages. This helped us understand the extent of missing data and decide on the necessary imputation strategies.

- **Missing Data Detected**:
    - **Numerical Columns**: Columns such as Age, RoomService, FoodCourt, etc., contained missing values.
    - **Categorical Columns**: Columns like HomePlanet, CryoSleep, Destination, and VIP also had missing entries.

```
Missing Values in the Dataset:
PassengerId       0
HomePlanet      201
CryoSleep       217
Cabin           199
Destination     182
Age             179
VIP             203
RoomService     181
FoodCourt       183
ShoppingMall    208
Spa             183
VRDeck          188
Name            200
Transported       0
dtype: int64

Percentage of Missing Values in Each Column:
PassengerId     0.000000
HomePlanet      2.312205
CryoSleep       2.496261
Cabin           2.289198
Destination     2.093639
Age             2.059128
VIP             2.335212
RoomService     2.082135
FoodCourt       2.105142
ShoppingMall    2.392730
Spa             2.105142
VRDeck          2.162660
Name            2.300702
Transported     0.000000
dtype: float64
```

Decision:
We chose to **impute** missing numerical values (e.g., Age, RoomService) with the **median**, as it is more robust to outliers. For categorical columns (e.g., HomePlanet, CryoSleep, VIP), we imputed the missing values with the **mode** (the most frequent value).

Here We Handled missing values in this way-

```python
print("Handling Missing Values...")

# Impute numerical columns with their median
numerical_cols_to_impute = ['Age', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']
for col in numerical_cols_to_impute:
    median_val = train[col].median()
    train[col] = train[col].fillna(median_val)
    print(f"  - Missing values in '{col}' imputed with median: {median_val}")

# Impute categorical columns with their mode
categorical_cols_to_impute = ['HomePlanet', 'CryoSleep', 'Destination', 'VIP']
for col in categorical_cols_to_impute:
    mode_val = train[col].mode()[0] # .mode() can return multiple if frequencies are tied
    train[col] = train[col].fillna(mode_val)
    print(f"  - Missing values in '{col}' imputed with mode: {mode_val}")
```

And the result is:

```
Handling Missing Values...
  - Missing values in 'Age' imputed with median: 27.0
  - Missing values in 'RoomService' imputed with median: 0.0
  - Missing values in 'FoodCourt' imputed with median: 0.0
  - Missing values in 'ShoppingMall' imputed with median: 0.0
  - Missing values in 'Spa' imputed with median: 0.0
  - Missing values in 'VRDeck' imputed with median: 0.0
  - Missing values in 'HomePlanet' imputed with mode: Earth
  - Missing values in 'CryoSleep' imputed with mode: False
  - Missing values in 'Destination' imputed with mode: TRAPPIST-1e
  - Missing values in 'VIP' imputed with mode: False
  - 'Deck' column extracted from 'Cabin', missing decks imputed with 'Unknown'.
  - Original 'Cabin' column dropped.
  - Replaced 200 missing values in 'Name' with 'Unknown'.

Verifying no more missing values after imputation:
Series([], dtype: int64)
/tmp/ipython-input-2247659198.py:14: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill
  train[col] = train[col].fillna(mode_val)
```

3.3.2 Checking for Duplicates (Rows and Columns)

We checked for both **duplicate rows** and **duplicate columns** in the dataset to ensure there was no redundant data.

Checking for Duplicate Rows

We used `train.duplicated().sum()` to verify if there were any exact duplicate rows in the dataset.

```
# Checking for duplicate rows
duplicates = train.duplicated().sum()
print(f"Duplicate Rows: {duplicates}")
```

- **Finding**: No duplicate rows were found.
- **Decision**: Since there were no duplicate rows, no further action was needed.

Checking for Duplicate Columns

We also ensured there were no duplicate columns by checking if any columns contained identical data.

- **Finding**: No duplicate columns were found.
- **Decision**: Since there were no duplicate columns, no further action was needed.

3.3.3 Inconsistent Data Types and Handling

We examined the **data types** of each column using `train.dtypes` to identify any inconsistencies in data types, particularly for columns like **CryoSleep** and **VIP**.

```
# Checking for data types
print(train.dtypes)
```

- **Finding**: Columns like **CryoSleep** and **VIP** were of **object** type due to `True`, `False`, and `NaN` values.
- **Decision**: We handled this inconsistency by imputing the **NaN** values in these columns with the **mode** (`False`), ensuring the columns were ready for analysis.

3.3.4 Handling Inconsistent Categories

We checked for **inconsistent categories** in key categorical columns such as **CryoSleep**, **HomePlanet**, and **Destination** to detect any potential errors or unexpected values.

```
# Checking unique values in key categorical columns
print(train['CryoSleep'].unique())
```

- **Finding**: The **CryoSleep** column had `NaN` values, but no major inconsistencies were found in the categorical data.
- **Decision**: We handled the `NaN` values by imputing them with the **mode** (most frequent value) for consistency.

4. Outlier Handling:

This phase focused on identifying and mitigating the impact of extreme values, or outliers, within the numerical features of the dataset. Outliers can disproportionately influence statistical analysis and machine learning models, leading to biased results or reduced model performance.

4.1 Identifying Outliers (IQR Method)

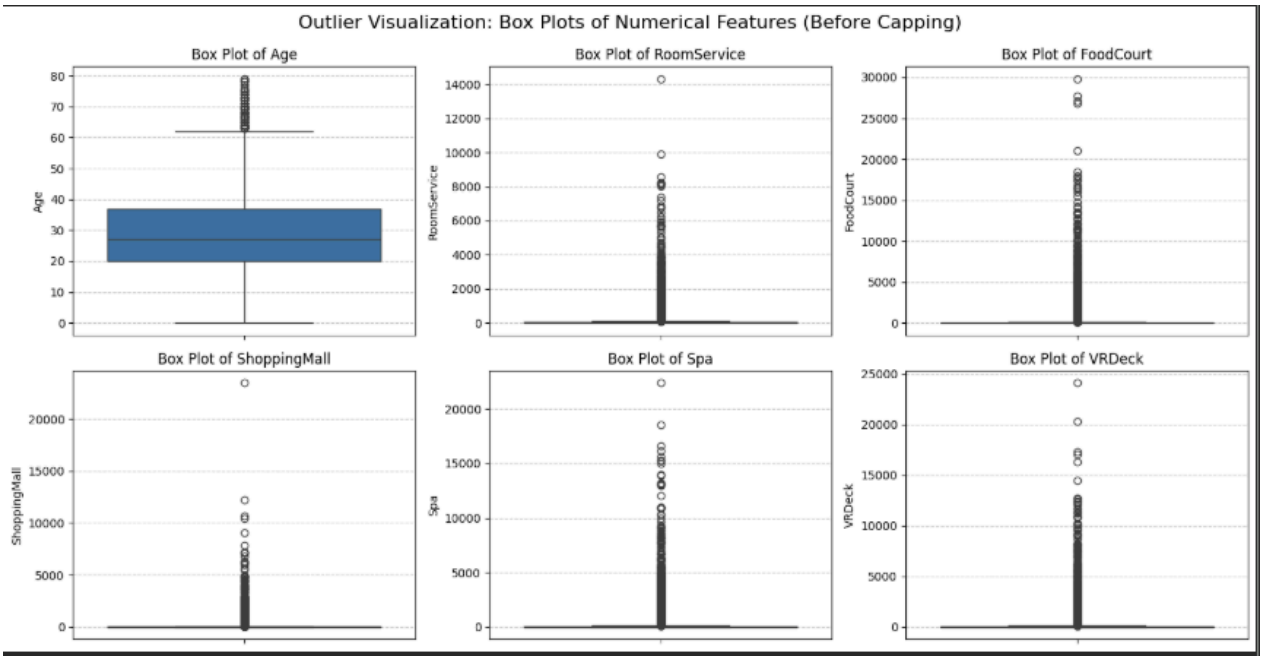
For numerical columns (Age, RoomService, FoodCourt, ShoppingMall, Spa, VRDeck), outliers were identified using the Interquartile Range (IQR) method. This involved calculating the first quartile (Q1), the third quartile (Q3), and the IQR (Q3 - Q1). Data points falling below Q1 - 1.5 * IQR or above Q3 + 1.5 * IQR were flagged as outliers.

Decision: The IQR method was chosen for its robustness, as it is less sensitive to extreme values than methods that rely on the mean and standard deviation. This method provided a clear, statistically sound basis for defining outliers.

4.2 Visualizing Outliers

After identifying outliers using the **IQR method**, we visualized the **distribution** of the numerical features to further confirm the presence and severity of outliers. **Box plots** were used as they effectively show the spread of data, highlighting values that fall outside the typical range (the outliers).

The box plots generated for the numerical features (Age, RoomService, FoodCourt, ShoppingMall, Spa, VRDeck) provide a clear visual representation of their distributions and the presence of outliers before any capping was applied.



4.3 Handling Outliers (Capping)

The identified outliers were addressed by capping their values. Specifically, values below the 5th percentile were replaced with the 5th percentile value, and values above the 95th percentile were replaced with the 95th percentile value for each respective numerical column.

- **Process**:
  - **Step 1**: We define the list of numerical columns to cap (e.g., `Age`, `RoomService`, `FoodCourt`, etc.).
  - **Step 2**: For each column, we calculate the **5th percentile** (lower bound) and **95th percentile** (upper bound).
  - **Step 3**: We apply **capping** using the `clip()` method, restricting values outside this range to the nearest boundary.

- **Step 4**: We verify that the values have been capped by printing the **minimum** and **maximum** values of each column after capping.

```
--- Addressing Outliers (Capping at 5th and 95th Percentiles) ---
  - Column 'Age': Values capped between 4.00 and 55.00.
  - Column 'RoomService': Values capped between 0.00 and 1256.32.
  - Column 'FoodCourt': Values capped between 0.00 and 2669.16.
  - Column 'ShoppingMall': Values capped between 0.00 and 912.16.
  - Column 'Spa': Values capped between 0.00 and 1573.28.
  - Column 'VRDeck': Values capped between 0.00 and 1479.48.

--- Verification: Min/Max values after capping ---
  Column Age: Min=4.00, Max=55.00
  Column RoomService: Min=0.00, Max=1256.32
  Column FoodCourt: Min=0.00, Max=2669.16
  Column ShoppingMall: Min=0.00, Max=912.16
  Column Spa: Min=0.00, Max=1573.28
  Column VRDeck: Min=0.00, Max=1479.48
```

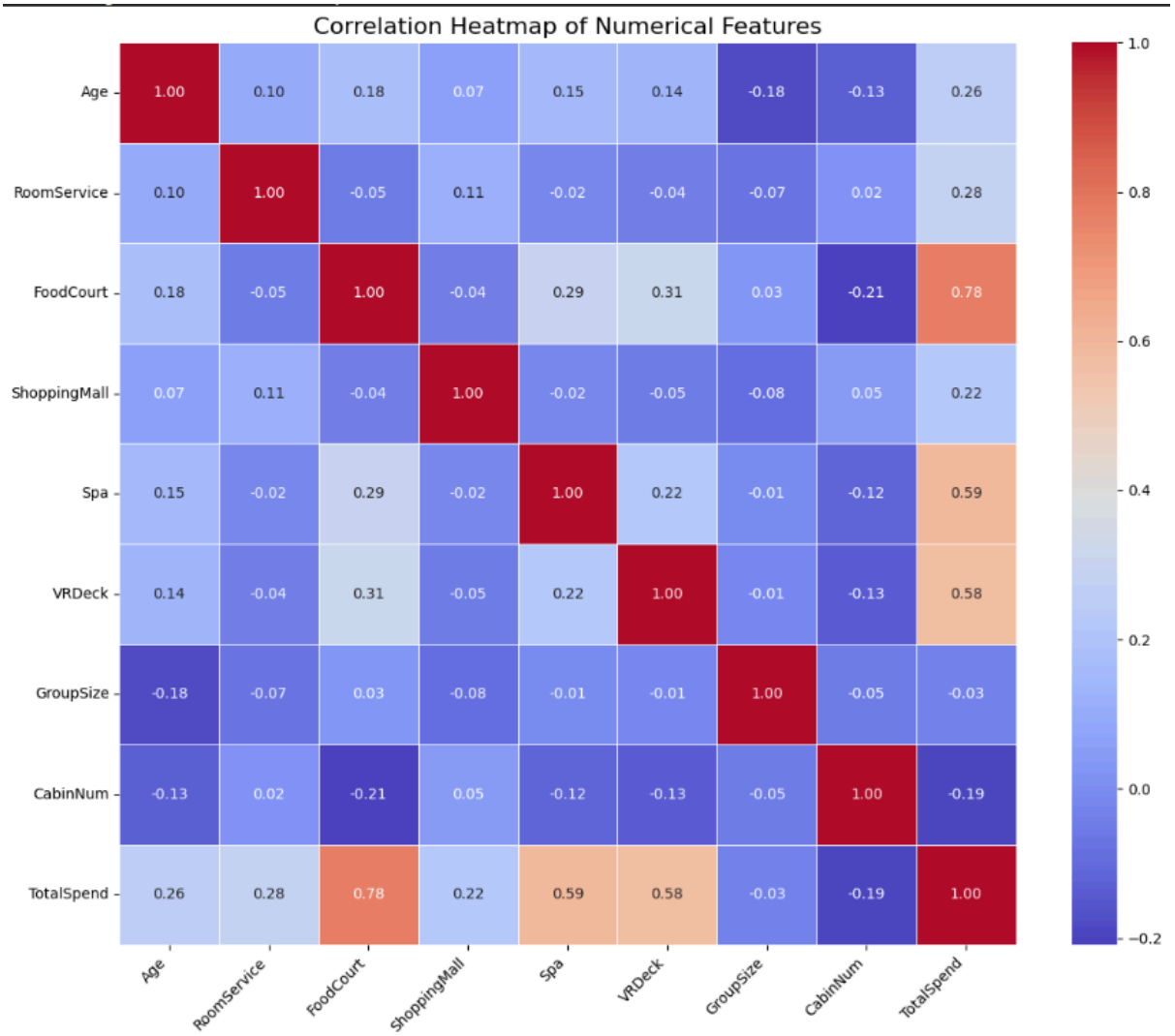
- **Decision**:
  - Capping outliers ensures that extreme values are brought within a reasonable range, reducing their impact on statistical analysis and machine learning models. This allows us to retain valuable data points while minimizing the bias caused by outliers.

5. Correlation Analysis:

To understand the linear relationships between numerical features, a correlation analysis was performed. This helps in identifying features that move together, which can be crucial for feature selection, dimensionality reduction, and model interpretation.

Method Used: seaborn.heatmap() with pandas.DataFrame.corr()

Description: A correlation matrix was calculated for all numerical features, including the original ones (Age, RoomService, FoodCourt, ShoppingMall, Spa, VRDeck) and the newly engineered numerical features (GroupSize, CabinNum, TotalSpend). This matrix was then visualized using a heatmap, with annotations showing the correlation coefficients.



Decisions/Insights:

Strong positive correlations were observed among the individual expense features and TotalSpend, as expected. This confirmed TotalSpend effectively summarizes these categories.

Weak to moderate correlations were noted between Age and certain expense features (e.g., FoodCourt, Spa).

GroupSize and CabinNum showed relatively weak correlations with other numerical features, suggesting they capture distinct information.

This analysis provided insights into potential multicollinearity (especially among expense features) and guided future feature selection decisions.

6. Feature Selection:

Feature selection is the process of identifying and retaining the most relevant features for building the predictive model. It helps improve model performance, reduces the risk of overfitting, and enhances the interpretability of the results. Below are the steps taken during feature selection and the rationale behind the decisions made.

What We Did:

- **Dropped `PassengerId` and `Name`**: These columns represent **unique identifiers** and do not provide any predictive value. Hence, they were removed to reduce noise and prevent overfitting.

- **Dropped `OriginalCabin`**: The `Cabin` column had high cardinality and contained complex string values. We extracted more granular features like **Deck**, **CabinNum**, and **Side** from the `Cabin` column, which provided more meaningful information. The original `Cabin` column was removed to avoid redundancy.

- **Kept Engineered Features**: We created new features, including:

    - `Group`: Categorical feature used for **one-hot encoding** to represent groups of passengers.

    - `GroupSize`: Numerical feature representing the size of the passenger's group.

    - `CabinNum`: Numerical feature representing the cabin number, which captures more granular information than the original `Cabin` column.

    - `Side`: Categorical feature representing the side of the spaceship (Port or Starboard), derived from the `Cabin` column.

    - `TotalSpend`: Numerical feature consolidating spending across multiple amenities (e.g., RoomService, FoodCourt, etc.) to measure a passenger's total spending.

Decisions/Rationale:

- **Removing `PassengerId` and `Name`**: These columns were dropped because they are unique identifiers and are not useful in making predictions. Retaining

such features can lead to **overfitting** since they do not add any predictive power to the model.

- **Decomposing `Cabin` into `Deck`, `CabinNum`, and `Side`**: The original `Cabin` column was complex and contained high cardinality (many unique values). By extracting structured features like **Deck**, **CabinNum**, and **Side**, we were able to capture more **interpretable and meaningful information** from the `Cabin` column. This transformation made it easier to use these features in modeling without the redundancy of the original column.

- **Creating `TotalSpend`, `Group`, and `GroupSize`**: These engineered features were created to capture **social dynamics** (e.g., Group) and **spending behavior** (e.g., TotalSpend), which are likely to have a strong impact on passenger outcomes. The new features were designed to provide the model with a richer and more nuanced understanding of the passengers' behaviors and relationships.

7. Data Splitting:

To rigorously evaluate the performance of our machine learning model and ensure its ability to generalize to unseen data, we split the dataset into distinct **training** and **validation** sets. This ensures that the model can be tested on data it hasn't seen during training, providing a more accurate estimate of its performance.

Method Used: `sklearn.model_selection.train_test_split`

What We Did:

1. The **preprocessed and feature-engineered** dataset (`train` DataFrame) was split into:
    - **X**: The feature columns (all columns except the target variable, `Transported`).
    - **y**: The target variable, `Transported`, which we aim to predict.
2. The dataset was then divided into **training** and **validation** sets using an **80/20 ratio**:
    - **Training Set**: 80% of the data used to train the model (`X_train`, `y_train`).

- **Validation Set**: 20% of the data reserved for evaluating the model (`X_test`, `y_test`).

3. A `random_state=42` was set to ensure the split is reproducible each time the code is run, providing consistency across different runs of the model.

4. **Stratified splitting** was applied by using `stratify=y`. This ensured that the proportion of **Transported (True/False)** passengers was maintained consistently across both the **training** and **validation** sets, preventing any class imbalance issues.

```
from sklearn.model_selection import train_test_split

# Split the data into features (X) and target variable (y)
X = train.drop(columns=['Transported'])
y = train['Transported']

# Split the data into training and validation sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
... Splitting the dataset into training and validation sets...
      - X (features) shape: (8693, 6241)
      - y (target) shape: (8693,)
    Dataset split successfully.

    Shapes of the resulting sets:
      - X_train shape: (6954, 6241)
      - y_train shape: (6954,)
      - X_test shape: (1739, 6241)
      - y_test shape: (1739,)
```

Decision/Rationale:

- **Training and Validation Split**: This step is critical for unbiased model evaluation. By separating the data into distinct sets, we ensure that the model is trained on one portion of the data and evaluated on a completely separate, unseen portion. This gives us a better understanding of how the model will perform on new, real-world data.

- **Stratified Split**: Stratification was essential to preserve the **class distribution** of the target variable (`Transported`) in both the training and

validation sets. This ensures that the proportion of **True/False** values in the target variable is similar across both sets, preventing any potential **class imbalance issues** that could negatively affect model training and evaluation.

8. Feature Scaling:

Feature scaling is a critical preprocessing step, particularly for machine learning algorithms that are sensitive to the magnitude and range of numerical features. Scaling ensures that each feature contributes equally to the model, avoiding dominance by features with larger values or ranges.

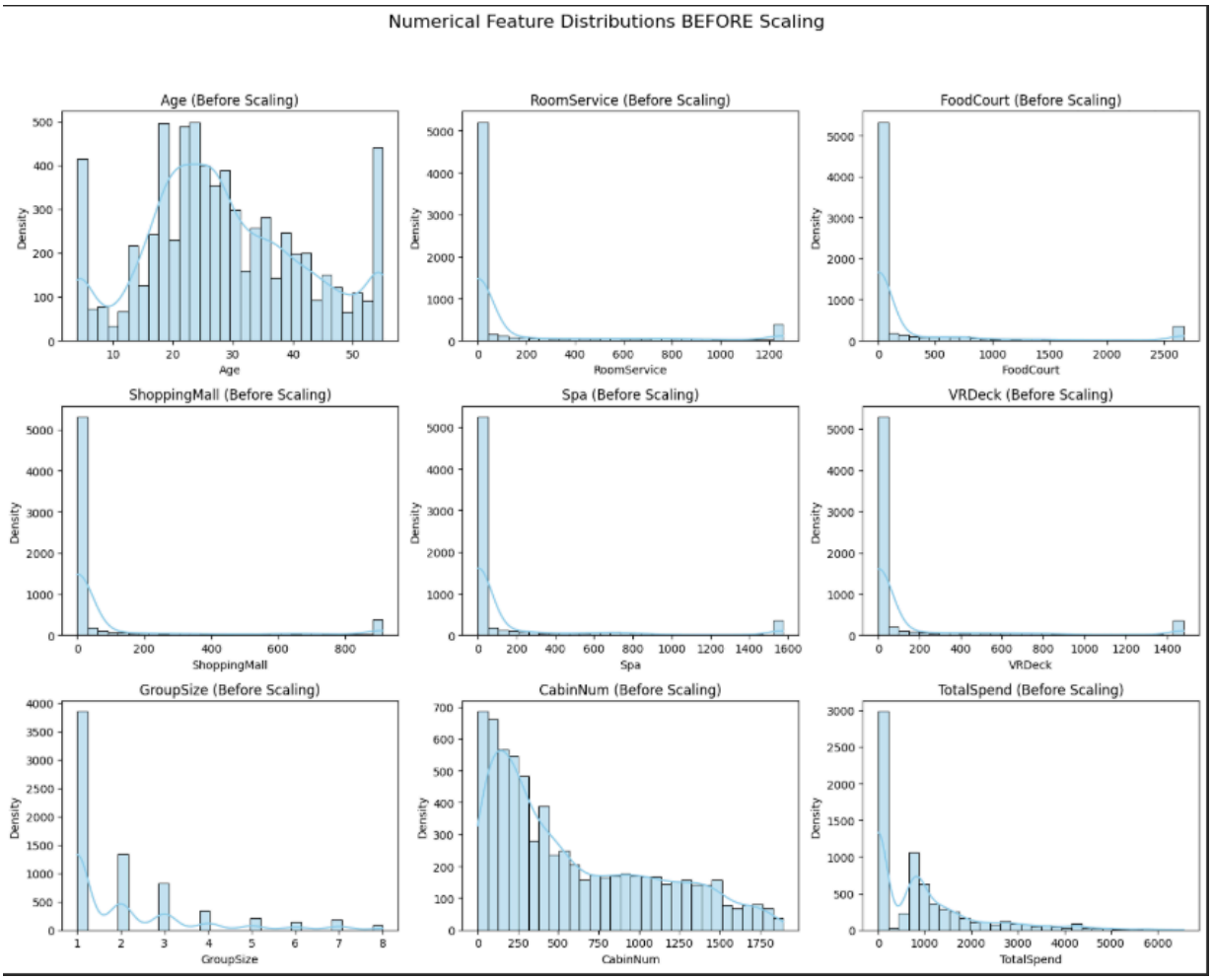
Method Used: `sklearn.preprocessing.StandardScaler`

What We Did:

1. **Numerical Columns Identified for Scaling**: The numerical columns identified for scaling were:
    - **Age**, **RoomService**, **FoodCourt**, **ShoppingMall**, **Spa**, **VRDeck**, **GroupSize**, **CabinNum**, **TotalSpend**.
2. **StandardScaler Fitting**: A `StandardScaler` was instantiated and fitted exclusively on the **training data (X_train)**. This step calculates the **mean** and **standard deviation** for each feature in the training set.
3. **Transformation**: The same fitted `StandardScaler` was then used to **transform** both the **training set (X_train)** and the **validation set (X_test)** numerical features, ensuring consistent scaling across both sets.
4. **Visualizing the Effect of Scaling**: Histograms were generated to show the distributions of the numerical features before and after scaling. This clearly demonstrated how the scaling process transformed the features to have a **mean of 0** and a **standard deviation of 1**.

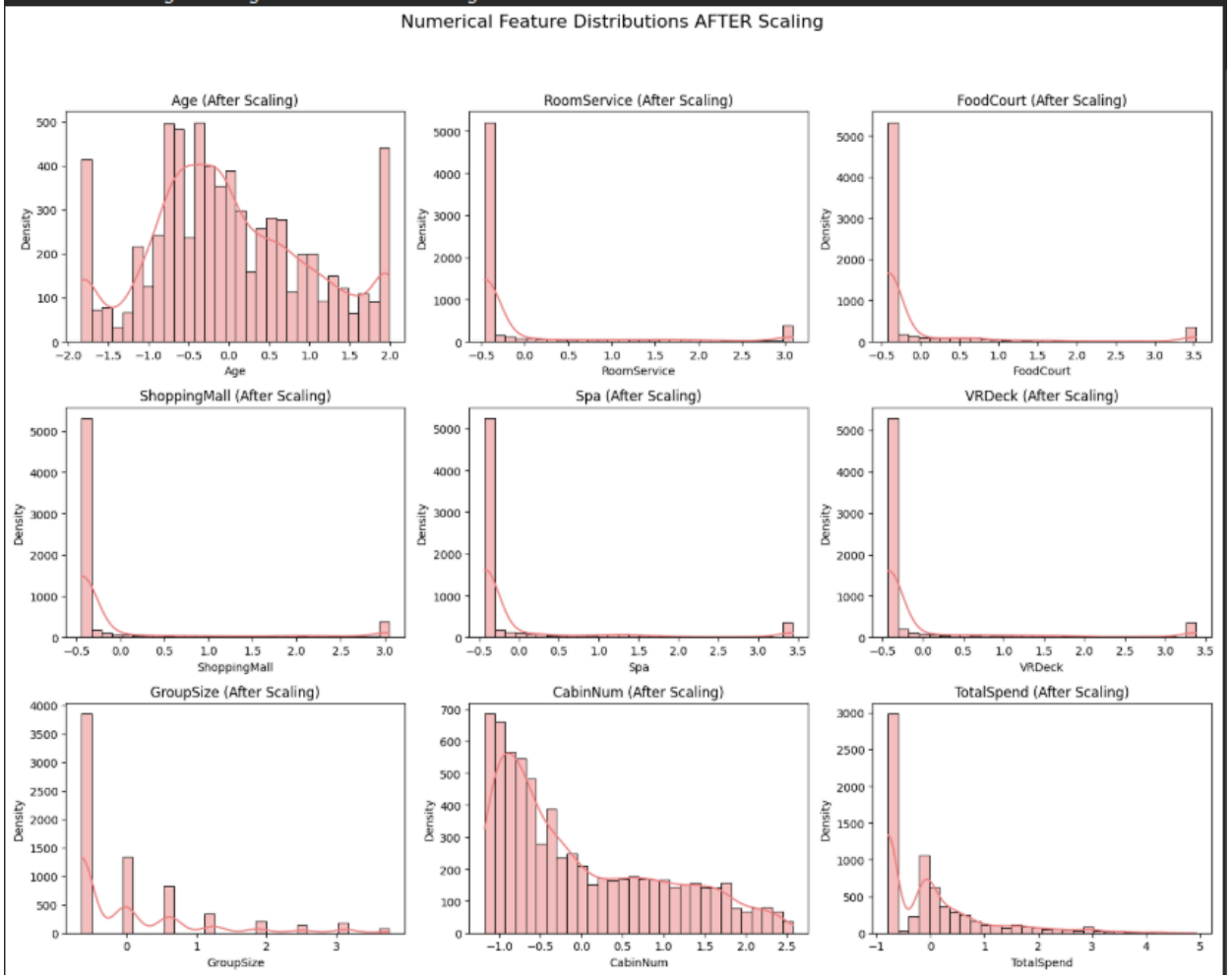
```
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Identify numerical columns for scaling
numerical_cols_to_scale = ['Age', 'RoomService', 'FoodCourt',
'ShoppingMall', 'Spa', 'VRDeck', 'GroupSize', 'CabinNum',
'TotalSpend']
```

```
# Initialize StandardScaler
scaler = StandardScaler()

# Fit and transform on training data
X_train_scaled = X_train[numerical_cols_to_scale].copy()
X_train_scaled = scaler.fit_transform(X_train_scaled)

# Transform on test data (using the same scaler)
X_test_scaled = X_test[numerical_cols_to_scale].copy()
X_test_scaled = scaler.transform(X_test_scaled)

# Visualizing the scaling effect with histograms
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Before scaling
axes[0].hist(X_train[numerical_cols_to_scale].values.flatten()
, bins=50)
axes[0].set_title('Before Scaling')
```



```
# After scaling
axes[1].hist(X_train_scaled.flatten(), bins=50)
axes[1].set_title('After Scaling')

plt.show()
```

```
- Applying StandardScaler to numerical columns...
  - X_train numerical columns scaled.
  - X_test numerical columns scaled using the same scaler.
- Generating histograms AFTER scaling...
```



9.Final Findings:

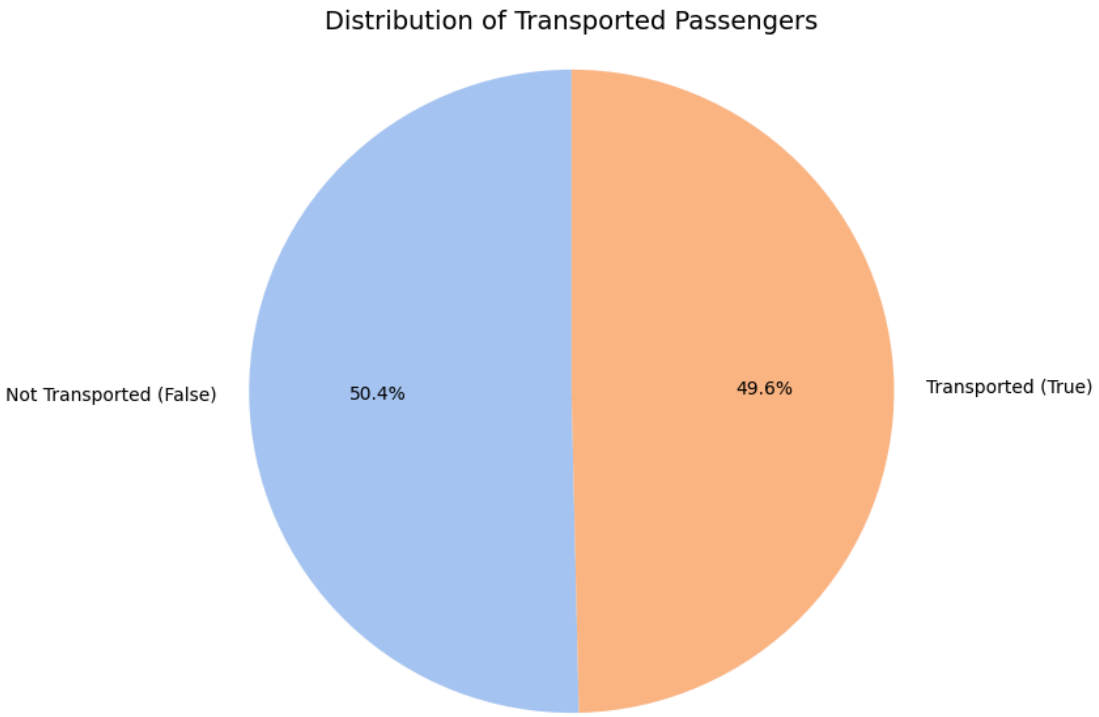
After performing the various **data preprocessing** steps, **outlier handling**, and **feature selection**, we arrived at a clean and ready-to-use dataset for modeling. Below is a summary of the key actions performed and the results:

Step	Action Taken	Result/ Output
Handling Missing Values	Imputed missing values in numerical columns with the **median** and in categorical columns with the **mode**.	All missing values were successfully filled, ensuring no missing data.
Checking for Duplicates	Checked for duplicate rows and columns.	No duplicate rows or columns were found.
Handling Inconsistent Data Types	Converted `CryoSleep` and `VIP` columns from object type to boolean by filling `NaN` values with **False** (mode).	Data types were corrected for proper processing.
Handling Inconsistent Categories	Inspected unique values in categorical columns and handled `NaN` values.	No major inconsistencies were found; `NaN` values were imputed correctly.
Outlier Handling	Capped values at the **5th and 95th percentiles** for numerical features.	Extreme outliers were capped, reducing their impact on the dataset.

Feature Selection	Removed irrelevant features like `PassengerId` and `Name`, and retained engineered features like `Group`, `GroupSize`, `TotalSpend`, and others.	The dataset was streamlined with relevant features for modeling.
Data Splitting	Split the dataset into **80% training** and **20% validation** using stratified sampling.	The data was split while preserving the target variable distribution.
Feature Scaling	Applied **StandardScaler** to scale numerical features to a mean of 0 and standard deviation of 1.	Numerical features were normalized, improving model performance.
Final Dataset Saved	Saved the final processed dataset to a CSV file named **train_processed.csv**.	The cleaned and preprocessed dataset was saved successfully for future use.

10.Pie Chart of 'Transported' Distribution:

Now, we will visualize the distribution of the target variable **`Transported`** to understand the balance between passengers who were **transported** and those who were **not transported**. Here's the pie chart:



11.Final Processed Dataset:

After completing all preprocessing steps, including handling missing values, detecting and handling outliers, and scaling features, the final processed dataset was saved to a new CSV file for further analysis and model building.

```
# Saving the final processed DataFrame to
'train_processed.csv'
train.to_csv('train_processed.csv', index=False)
```

- **Result**: The processed dataset is now available as **train_processed.csv**, ready for modeling or further analysis.

12.Discussion:

The preprocessing steps outlined in this report were essential for transforming the **Spaceship Titanic** dataset into a clean and structured form suitable for machine learning modeling. The dataset contained both numerical and categorical features, and addressing issues like missing values, duplicates, outliers, and irrelevant features was crucial for ensuring that the model would perform effectively.

We began by handling missing values, which is a common issue in real-world datasets. For **numerical columns**, we chose to impute missing values with the **median**. This decision was based on the fact that the median is less sensitive to outliers than the mean and provides a robust estimate for missing values. For **categorical columns**, we imputed missing values with the **mode**, the most frequent category, which is a standard practice for dealing with missing categorical data. This approach allowed us to preserve the structure of the data while filling in gaps.

Next, we checked for and handled **duplicate rows** and **duplicate columns**. Fortunately, the dataset contained no duplicate rows or columns, which meant that no further action was needed. This step ensured that there was no redundant data that could introduce bias or overfitting during modeling.

Inconsistent data types were then addressed. We identified that some columns, such as **CryoSleep** and **VIP**, were stored as object types due to the presence of boolean values and `NaN` entries. These columns were corrected by imputing missing values and converting the data types to boolean. This step ensured that the dataset was properly formatted for machine learning algorithms.

Outliers, which can have a disproportionate effect on model performance, were handled next. Using the **Interquartile Range (IQR)** method, we identified and capped extreme values in numerical features like **Age**, **RoomService**, and **Spa**. By applying capping at the **5th and 95th percentiles**, we reduced the influence of outliers without discarding valuable data points. This process helps to avoid skewing the model by extreme values that don't represent the general trends in the data.

Feature selection was another important step, where we removed non-predictive columns like **PassengerId** and **Name**, which are unique identifiers and carry no predictive power. We also decomposed the **Cabin** column into more granular features, such as **Deck**, **CabinNum**, and **Side**, to capture more meaningful patterns from the original complex data. The creation of **TotalSpend**, combining various spending columns, and keeping engineered features like **Group** and **GroupSize** ensured that the dataset contained features that provided a richer understanding of the passengers' behaviors.

Data splitting was performed using an **80/20** ratio, ensuring that **stratified sampling** was applied to preserve the distribution of the target variable **Transported**. This was crucial for ensuring that the validation set had the same class distribution as the training set, preventing any bias that might arise from class imbalance.

Feature scaling was applied using **StandardScaler**, which transformed the numerical features to have a mean of 0 and a standard deviation of 1. This step was vital for models that are sensitive to the scale of input features, such as **KNN**, **SVM**, and gradient descent-based models. Scaling ensures that no feature dominates the learning process due to its larger magnitude.

Finally, after all the preprocessing steps were completed, the cleaned dataset was saved as **train_processed.csv**. This final dataset is now ready for use in machine learning models. Saving the dataset ensures that all preprocessing steps are preserved and can be reused or shared for further analysis.

In conclusion, the preprocessing steps were essential for transforming the raw dataset into a form that is suitable for machine learning. Each decision, from imputing missing values to handling outliers and scaling features, was made with the goal of improving the dataset’s quality and the model's ability to learn meaningful patterns. The processed dataset is now clean, structured, and ready for modeling, and the decisions made during the preprocessing phase align with industry best practices to ensure the success of the subsequent model development.